PHISHSHIELD | Cybersecurity Interview Master Cheat Sheet

WEB APPLICATION
Rules (65%) + ML (35%) | Flask & React 19

1. THE 30-SECOND PITCH (What is your project?)

'PhishShield' is a web-based **phishing detection and threat analysis system**. It allows users to paste a suspicious website link (URL) or email to check if it is safe or a phishing attack. It uses a **hybrid detection engine** that combines cybersecurity rules with Scikit-Learn Machine Learning to produce an accurate, easy-to-understand risk score from 0 to 100.

2. PURPOSE (Why did you build it?)

• **Problem:** Phishing attacks cause over 90% of corporate data breaches. Regular users and security analysts need a fast, simple tool to verify suspicious messages.
• **Solution:** PhishShield gives immediate verdicts (Safe, Suspicious, or Phishing), explains the exact technical reasons why, and lets users export PDF reports.

3. TECHNOLOGIES USED (Verified Stack)

• **Frontend:** React 19, Vite 8, Tailwind CSS, Lucide React Icons.
• **Charts:** Recharts (Donut chart & risk volume charts).
• **Backend:** Python 3.13, Flask 3.1 REST API, Flask-CORS.
• **Machine Learning:** Scikit-Learn (Random Forest for URLs, TF-IDF + Logistic Regression for Email NLP).
• **Database:** SQLite 3 (stores scan history and stats).
• **Reporting:** jsPDF & AutoTable (generates instant PDF reports).
• **Testing:** Pytest (23 automated test cases covering all analyzers).

4. WHY IS IT CALLED 'HYBRID'?

Because it combines two things: **Rules (65%)** catch obvious technical tricks (like fake domains and IP hosts), while **Machine Learning (35%)** catches subtle text patterns. Together, they are much more reliable than using either one alone.

5. APPLICATION WORKFLOW (Step-by-Step)

Step 1 (Input): User pastes a suspicious URL or raw email text into the React frontend.
Step 2 (Rule Check): Backend checks 14 URL rules (IP address hosts, typosquatting, suspicious TLDs) or 7 email rules (Reply-To mismatch, urgent language).
Step 3 (ML Check): Scikit-Learn models analyze URL structure (15 features) and email words (TF-IDF) to calculate an ML risk probability.
Step 4 (Final Score): The system calculates the score: Final Risk = 0.65 * Rules + 0.35 * ML.
Step 5 (Result): The user sees the Verdict (Safe < 35, Suspicious 35-69, Phishing ≥ 70), risk score out of 100, and explanation table.

6. IMPORTANT PRACTICAL NOTES FOR INTERVIEWS:

• **User Privacy:** Raw email bodies and passwords are never permanently saved in the SQLite database—only scan results and rule IDs are logged.
• **Safe URL Parsing:** The app inspects the URL's text structure without opening dangerous malicious websites in the backend.
• **Severity Floors:** If an obvious danger is detected (like an IP address host), the risk score automatically jumps to ≥ 75 (Phishing).
• **Real-Time Dashboard:** The dashboard updates scan statistics automatically and displays accurate time ('Just now', '1m ago').

7. TOP 5 INTERVIEW QUESTIONS & HOW TO ANSWER CONFIDENTLY

Q1: What does your application do?	**Answer:** PhishShield is a web application that scans suspicious URLs and emails. It uses a combination of cybersecurity rules and Scikit-Learn machine learning to detect phishing attacks and assign a risk score from 0 to 100.
Q2: Is this a rule-based or machine learning project?	**Answer:** It is a **hybrid system**. It uses rules (65% weight) to catch clear violations like fake domain names and raw IP addresses, and machine learning (35% weight) to catch subtle word patterns and complex URL structures.
Q3: What machine learning algorithms did you use?	**Answer:** I used Scikit-Learn's RandomForestClassifier for URLs using a 15-feature lexical vector, and TfidfVectorizer with LogisticRegression to analyze email text and detect phishing keywords.
Q4: How do you detect fake websites (Typosquatting)?	**Answer:** I use the Levenshtein distance algorithm. It compares the entered domain with popular brands (like PayPal, Google, Microsoft). If the name is only 1 character different (like paypa1.com), it flags it as typosquatting.
Q5: How is user data protected?	**Answer:** Privacy is built-in. When a user scans an email, the backend analyzes it in memory and immediately discards the raw body text. We only store the subject line, risk score, and timestamp in SQLite.